Autenticación, permisos y tokens

Los fragmentos conservan las llamadas del código; se omiten comentarios, registros de diagnóstico y partes de las funciones.

Inicio de sesión y permisos

src/gui/app.py

```
self.email_msal_app = PublicClientApplication(
    client_id=MICROSOFT_CLIENT_ID,
    authority='https://login.microsoftonline.com/common',
    token_cache=self.email_token_cache.cache
)
```

MSAL identifica la aplicación mediante el Client ID. El inicio de sesión ocurre en Microsoft, donde se aplican las políticas de acceso y MFA del tenant. Este cliente de escritorio no utiliza un secreto de aplicación.

Los tres permisos son **delegados**: el acceso se realiza en nombre del usuario autenticado.

Permiso	Servicio	Uso en la aplicación
`User.Read`	Microsoft Graph	Consultar nombre, correo y alias del usuario.
`Mail.Send`	Microsoft Graph	Enviar mensajes y adjuntos desde su buzón.
`AllSites.Read`	SharePoint	Descargar el archivo configurado. Permite leer otros sitios a los que el usuario tenga acceso; no queda limitado a ese archivo.

Autenticación interactiva

```
result = self.sharepoint_msal_app.acquire_token_interactive(
    scopes=scopes, prompt='select_account')
```

Esta llamada abre el navegador y MSAL incorpora PKCE, que vincula el canje del código de autorización con la solicitud que lo originó. Funcionamiento de MSAL.

Retorno del correo en la ventana de configuración

```
redirect_uri = "http://localhost:62190/"
# ... apertura del navegador y recepción del código ...
result = app.acquire_token_by_authorization_code(
    auth_code[0], scopes=scopes, redirect_uri=redirect_uri)
```

Microsoft devuelve un código que se intercambia por tokens. La contraseña no pasa por este manejador.

Este retorno manual no valida `state` ni configura PKCE. Además, el servidor usa `TCPServer(("", 62190), ...)`, que escucha en todas las interfaces. Para incorporar esas protecciones, debe usarse el flujo interactivo de MSAL y un retorno limitado al equipo local.

Perfil y envío de correo

src/gui/app.py

```
headers = {'Authorization': f'Bearer {access_token}'}
response = requests.get(
    'https://graph.microsoft.com/v1.0/me', headers=headers)
response.raise_for_status()
user_info = response.json()
email = user_info.get('mail') or user_info.get('userPrincipalName')
name = user_info.get('displayName')
```

`User.Read` permite identificar la cuenta conectada. Los alias se consultan con `/me?$select=proxyAddresses`. Estas llamadas consultan el perfil; no descargan mensajes del buzón.

src/email_service/email_sender.py

```
MS_SCOPES = ['User.Read', 'Mail.Send']

headers = {
    'Authorization': f'Bearer {access_token}',
    'Content-Type': 'application/json'
}
# ... construcción de message ...
response = requests.post(
    'https://graph.microsoft.com/v1.0/me/sendMail',
    headers=headers,
    json=message
)
```

El token viaja en la cabecera de una petición HTTPS a Microsoft Graph. El cuerpo contiene asunto, texto HTML, destinatarios, copias y adjunto. `Mail.Send` autoriza el envío; no concede lectura de correos. HTTP 202 significa que Microsoft aceptó la solicitud, no que el destinatario ya la recibió. Referencia de sendMail.

Archivo adjunto y remitente

```
with open(attachment_path, 'rb') as f:
    file_data = f.read()
    file_name = os.path.basename(attachment_path)
    encoded_content = base64.b64encode(file_data).decode()
    attachment = {
        "@odata.type": "#microsoft.graph.fileAttachment",
        "Name": file_name,
        "ContentBytes": encoded_content
    }
message["message"]["attachments"] = [attachment]
```

Se lee el archivo local indicado y se incorpora al mensaje. Base64 es una codificación, no cifrado; HTTPS protege el transporte hacia Microsoft.

El campo `from` puede indicar otra dirección, pero no otorga acceso a ese buzón. Para enviar desde un buzón ajeno o compartido se requiere adaptar los scopes a `Mail.Send.Shared` y disponer de la delegación correspondiente en Exchange. Permisos del remitente.

Lectura de archivos de SharePoint

src/gui/app.py

```
raw_site_url = self.config_manager.get_sharepoint_site_url()
from urllib.parse import urlparse
parsed_url = urlparse(raw_site_url)
base_domain = f"{parsed_url.scheme}://{parsed_url.netloc}"
scopes = [f"{base_domain}/.default"]
```

La dirección configurada determina el recurso para el que se pide el token. `/.default` solicita los permisos delegados configurados para SharePoint; en este registro se utiliza `AllSites.Read`. El usuario también debe tener acceso al archivo.

src/processing/file_processor.py

```
server_relative_path = quote(self.sharepoint_file_url)
endpoint = f"{self.site_url}/_api/web/GetFileByServerRelativeUrl('{server_relative_path}')/$value"
headers = {
    "Authorization": f"Bearer {access_token}",
    "Accept": "application/json;odata=nometadata"
}
response = requests.get(endpoint, headers=headers, stream=True)

if response.status_code == 200:
    with open(self.local_file_path, 'wb') as f:
        for chunk in response.iter_content(chunk_size=8192):
            f.write(chunk)
```

La petición GET descarga el contenido mediante la API REST de SharePoint. Esta operación no modifica ni sube archivos. La copia se guarda en `data/temp/` y puede reutilizarse en ejecuciones posteriores.

El token se envía a la URL configurada. Debe ser una dirección HTTPS del SharePoint autorizado; este fragmento no valida una lista de dominios. La copia descargada queda sujeta a los permisos del disco local.

Reutilización del token

src/email_service/email_sender.py

```
accounts = self.msal_app.get_accounts()
if accounts:
    result = self.msal_app.acquire_token_silent(
        self.MS_SCOPES, account=accounts[0])
    if result and 'access_token' in result:
        return result['access_token']
```

MSAL busca un token válido en la caché o intenta renovarlo. Si no obtiene uno, el envío requiere iniciar sesión otra vez. La renovación sigue sujeta a las políticas de Microsoft; cerrar el navegador no elimina los tokens guardados. Adquisición silenciosa de tokens.

Almacenamiento de tokens y cierre de sesión

src/gui/app.py

```
self.email_token_cache = SecureTokenCache(
    cache_name='email_token_cache')
self.sharepoint_token_cache = SecureTokenCache(
    cache_name='sharepoint_token_cache')
```

Correo y SharePoint mantienen cachés separadas. Cada una puede contener tokens de acceso y de renovación, por lo que su contenido debe tratarse como una credencial.

src/utils/token_cache.py

```
serialized_cache = self.cache.serialize()
# ... división en fragmentos ...
keyring.set_password('my_app', fragment_key, fragment)
```

La caché se guarda mediante `keyring`, que puede usar el almacén de credenciales de Windows. La protección depende del backend activo: el código no lo fuerza ni lo comprueba. Dividir la caché en fragmentos resuelve límites de tamaño, pero no añade cifrado. Almacenamiento con keyring.

```
fragment = keyring.get_password('my_app', fragment_key)
# ... unión de los fragmentos ...
self.cache.deserialize(serialized_cache)
```

Al abrir la aplicación, los fragmentos se recuperan y reconstruyen en memoria para que MSAL pueda reutilizar la sesión.

Cerrar sesión

src/gui/app.py

```
self.config_manager.clear_credentials()
self.email_token_cache.clear_cache()
self.sharepoint_token_cache.clear_cache()
```

Se eliminan los datos de sesión de la configuración y se solicita vaciar las dos cachés. En `clear_cache()` se utilizan estas llamadas:

```
self.cache.deserialize("")
# ... recorrido de los fragmentos almacenados ...
keyring.delete_password('my_app', fragment_key)
```

La limpieza afecta a la memoria y al almacén local. No revoca el consentimiento en Entra ni cierra la sesión del navegador. Tampoco borra archivos descargados o correos enviados. Si el almacén falla, la eliminación puede quedar incompleta.

Los registros de diagnóstico pueden contener cuentas, destinatarios, rutas y errores. El servidor del retorno manual también puede registrar la URL con el código de autorización; esos registros no deben compartirse sin depurarlos.